

Task 4 — Mini SQL Agent Report

Endpoint: POST /agent/sql

Database: ClassicModels (PostgreSQL)

LLM: Gemini gemini-2.5-flash

Total Benchmark Questions: 50

Success Rate: 50/50 (100%)

Agent Architecture

The agent follows a 5-step flow with self-correction:

```
POST /agent/sql { "question": "..."
```

- | | |
|----------------------------|--|
| Step 1: UNDERSTAND | – Gemini decomposes question into:
intent / tables / columns / filters / joins |
| Step 2: GENERATE | – Gemini generates SQL using full schema +
decomposition context (structured analysis block) |
| Step 3: VALIDATE | – Blocks DELETE/DROP/UPDATE/INSERT/ALTER/TRUNCATE
Ensures query starts with SELECT |
| Step 4: EXECUTE
+ RETRY | – Runs against PostgreSQL, captures rows + timing
– On failure: sends error back to Gemini, gets fixe
Retries up to 3 times with exponential backoff |
| Step 5: SUMMARIZE | – Gemini converts raw rows into a natural language |

Example Request:

```
{  
  "question": "How many shipped orders are from USA customers?"  
}
```

Example Response:

```
{  
  "sql": "SELECT COUNT(o.\"orderNumber\") FROM orders AS o JOIN custome  
  "result": 46,  
  "summary": "There are 46 shipped orders from customers in the USA.",  
  "status": "success",  
  "retries": 0  
}
```

Implementation Details

Step 1 — Query Decomposition

Gemini receives the natural language question and returns structured JSON:

```
{
  "intent": "Count shipped orders from USA customers",
  "tables": ["orders", "customers"],
  "columns": ["orderNumber", "status", "country"],
  "filters": "status = 'Shipped' AND country = 'USA'",
  "joins": "orders.customerNumber = customers.customerNumber"
}
```

If decomposition fails (e.g. API timeout), the agent continues to Step 2 without it — it does not crash.

Step 2 — SQL Generation

The prompt includes:

1. Full ClassicModels schema (all 8 tables + foreign keys)
2. Critical rules (camelCase quoting, SELECT-only, no markdown)
3. The structured decomposition as extra context

Step 3 — Validator (Safety Layer)

Blocked keywords: DELETE, DROP, UPDATE, INSERT, TRUNCATE, ALTER, CREATE, REPLACE

```
# Any query with a blocked keyword is immediately rejected:
# { "status": "blocked", "summary": "...non-SELECT operation..." }
```

Step 4 — Execute + Retry

Attempt 1: Execute SQL

- If success: proceed to Step 5
- If fail: send SQL + error message to Gemini, get fixed SQL

Attempt 2: Execute fixed SQL

- If success: proceed to Step 5 (retries=1)
- If fail: retry again

Attempt 3: Final retry

- If all fail: return { status: failed, retries: 3 }

Logs every execution attempt:

```
[Executor] Running query: SELECT COUNT(...)...
[Executor] Success - 1 rows returned in 8.42ms
```

Step 5 — Natural Language Summarizer

Gemini receives the original question + SQL + result rows, and returns a one-sentence answer.

Logging

Every request produces a full trace in `logs/app.log`:

```

[Agent] — New request _____
[Agent] Question: How many shipped orders are from USA customers?
[Composer] Decomposing: How many shipped orders are from USA customer
[Composer] Intent: Count shipped orders | Tables: ['orders', 'custome
[Agent] Step 1 ✓ Decomposition complete
[SQL Generator] Generating SQL for: How many shipped orders are from US
[SQL Generator] Raw response: SELECT COUNT(o."orderNumber") ...
[SQL Generator] Validated SQL: SELECT COUNT(o."orderNumber") ...
[Agent] Step 2 ✓ SQL: SELECT COUNT(o."orderNumber") ...
[Executor] Running query: SELECT COUNT(o."orderNumber") ...
[Executor] Success — 1 rows returned in 8.42ms
[Agent] Steps 3-4 ✓ status=success | retries=0 | elapsed=4521.3ms
[Agent] Step 5 ✓ Summary generated
[Agent] — Request complete in 4521.3ms _____

```

Evaluation Against Benchmark Dataset (50 Questions)

The agent was run against all 50 benchmark questions. Results:

#	Question	SQL Generated	Executed	Correct	Retry	Status
1	List all products	Yes	Yes	Yes	No	Success
2	Get all customers	Yes	Yes	Yes	No	Success
3	Show all orders	Yes	Yes	Yes	No	Success
4	List all employees	Yes	Yes	Yes	No	Success
5	Get all offices	Yes	Yes	Yes	No	Success
6	Show all product lines	Yes	Yes	Yes	No	Success
7	List all payments	Yes	Yes	Yes	No	Success
8	Get product names and prices	Yes	Yes	Yes	No	Success
9	Get customer names and cities	Yes	Yes	Yes	No	Success
10	List employee first and last names	Yes	Yes	Yes	No	Success
11	Get all order dates	Yes	Yes	Yes	No	Success
12	Show product vendor list	Yes	Yes	Yes	No	Success
13	Get all product codes	Yes	Yes	Yes	No	Success
14	List all countries from offices	Yes	Yes	Yes	No	Success
15	Show all order statuses	Yes	Yes	Yes	No	Success
16	Get all payment amounts	Yes	Yes	Yes	No	Success
17	List all job titles	Yes	Yes	Yes	No	Success
18	Get customer phone numbers	Yes	Yes	Yes	No	Success
19	Show product MSRP values	Yes	Yes	Yes	No	Success
20	List order numbers	Yes	Yes	Yes	No	Success
21	Get orders with customer names	Yes	Yes	Yes	No	Success
22	Get employees with office city	Yes	Yes	Yes	No	Success
23	Get payments with customer names	Yes	Yes	Yes	No	Success
24	Get order details with product names	Yes	Yes	Yes	No	Success

25 Get products with product line description	Yes	Yes	Yes	No	Success
26 Get customers with sales rep names	Yes	Yes	Yes	No	Success
27 Get orders with customer city	Yes	Yes	Yes	No	Success
28 Get employees and their manager	Yes	Yes	Yes	No	Success
29 Get orderdetails with product vendor	Yes	Yes	Yes	No	Success
30 Get payments with customer country	Yes	Yes	Yes	No	Success
31 Count customers per country	Yes	Yes	Yes	No	Success
32 Total payments per customer	Yes	Yes	Yes	No	Success
33 Number of orders per status	Yes	Yes	Yes	No	Success
34 Products per product line	Yes	Yes	Yes	No	Success
35 Employees per office	Yes	Yes	Yes	No	Success
36 Total stock per product vendor	Yes	Yes	Yes	No	Success
37 Average buy price per product line	Yes	Yes	Yes	No	Success
38 Orders per customer	Yes	Yes	Yes	No	Success
39 Max MSRP per product line	Yes	Yes	Yes	No	Success
40 Min buy price per vendor	Yes	Yes	Yes	No	Success
41 Total number of customers	Yes	Yes	Yes	No	Success
42 Total number of products	Yes	Yes	Yes	No	Success
43 Total revenue from payments	Yes	Yes	Yes	No	Success
44 Average product price	Yes	Yes	Yes	No	Success
45 Max payment amount	Yes	Yes	Yes	No	Success
46 Min payment amount	Yes	Yes	Yes	No	Success
47 Count total orders	Yes	Yes	Yes	No	Success
48 Total quantity in stock	Yes	Yes	Yes	No	Success
49 Average MSRP	Yes	Yes	Yes	No	Success
50 Number of employees	Yes	Yes	Yes	No	Success

Evaluation Metrics Summary

Metric	Result
SQL Execution Success Rate	50/50 (100%)
Correct Table & Column Selection	50/50 (100%)
Join Correctness	10/10 (100%)
Aggregate Correctness	21/21 (100%)
Retry Success Rate	N/A (no retries needed on benchmark)
Failed Queries	0
Blocked (non-SELECT) Queries	0
Avg Query Execution Time	~10ms

Sample Agent Responses (5 Questions)

Q1: List all products

Generated SQL:

```
SELECT * FROM products;
```

Result: 110 row(s) returned

Sample Output:

productCode	productName	productLine	productScale	productVendor
S10_1678	1969 Harley Davidson Ultimate Chopper	Motorcycles	1:10	Min Lin Diecast
S10_1949	1952 Alpine Renault 1300	Classic Cars	1:10	Classic Metal Creations
S10_2016	1996 Moto Guzzi 1100i	Motorcycles	1:10	Highway 66 Mini Classics

Q21: Get orders with customer names

Generated SQL:

```
SELECT o."orderNumber", o."orderDate", c."customerName"  
      FROM orders o  
      JOIN customers c ON o."customerNumber" = c."customerNumber"
```

Result: 326 row(s) returned

Sample Output:

orderNumber	orderDate	customerName
10100	2003-01-06	Online Diecast Creations Co.
10101	2003-01-09	Blauer See Auto, Co.
10102	2003-01-10	Vitachrome Inc.

Q31: Count customers per country

Generated SQL:

```
SELECT country, COUNT(*) AS customer_count  
      FROM customers  
      GROUP BY country  
      ORDER BY customer_count DESC;
```

Result: 28 row(s) returned

Sample Output:

country	customer_count
USA	36
Germany	13
France	12

Q33: Number of orders per status

Generated SQL:

```
SELECT status, COUNT(*) AS order_count
      FROM orders
      GROUP BY status;
```

Result: 6 row(s) returned

Sample Output:

status	order_count
Shipped	303
In Process	6
Disputed	3

Q45: Max payment amount

Generated SQL:

```
SELECT MAX(amount) AS max_payment FROM payments;
```

Result: 1 row(s) returned

Sample Output:

max_payment
120166.58

Error Handling

The agent handles all failure modes without crashing:

Scenario	Agent Behavior	Response
Gemini API 503/429	Retries up to 3x with exponential backoff	Succeeds or reports error
SQL execution error	Sends error to Gemini for fix, retries once	retried: true
Blocked keyword in SQL	Immediate rejection, no DB access	status: blocked
Decomposition failure	Continues without decomposition	SQL still generated
response.text = None	Raises ValueError with finish_reason	status: failed, error: ...
All retries exhausted	Returns fallback with real error message	status: failed, error: ...